

CHAIN-ANALYSIS

Blockchain Transaction Analysis for Anti-Money Laundering

System Overview & Development Guide

Version 1.0

February 2026

1. Executive Summary

Chain-Analysis is an internal blockchain transaction analysis platform designed for detecting and investigating money laundering patterns on Ethereum and EVM-compatible chains. The system models blockchain entities (wallets, contracts, protocols) as a property graph, enables human analysts to label suspicious activity, and provides a visual interface for exploring transaction flows and graph patterns.

The platform ingests transaction data from Allium (primary) with Ethereum node and Etherscan as fallback sources, stores entity relationships in Neo4j for graph traversal and algorithm execution, and uses PostgreSQL for labeling workflows and operational metadata. A React-based frontend provides interactive graph visualization, annotation tools, and analytical dashboards.

At production scale (5TB+ raw data), the system employs a hybrid Rust/Python architecture: Rust handles CPU-intensive ETL ingestion, feature computation, and custom graph algorithms, while Python manages the API layer, orchestration, and ad-hoc analysis. For production deployment, managed AWS services (Neo4j Aura, RDS Aurora, ECS Fargate, S3 + Athena) replace self-hosted containers, with a three-tier storage strategy that keeps Neo4j lean for interactive analysis while the full dataset lives in cheaper storage.

2. Architecture Overview

2.1 High-Level Architecture

The system follows a layered architecture with clear separation between data ingestion, storage, analysis, and presentation:

Layer	Components	Purpose
Ingestion	Allium SQL, Etherscan API, Erigon Node	Raw blockchain data acquisition and normalization
Pipeline	Python ETL (Dagster), Kafka/Redis Streams	Decode, resolve entities, compute features, stream to storage
Storage	Neo4j (graph), PostgreSQL (relational)	Entity graph + labeling/metadata
API	FastAPI (Python)	REST/GraphQL endpoints for frontend and batch jobs
Frontend	React + TypeScript, Cytoscape.js	Graph visualization, annotation UI, dashboards

2.2 Data Flow

The end-to-end data flow from raw blockchain data to analyst-facing interface:

1. Allium SQL queries extract transactions, traces (internal transactions), and decoded events for target block ranges.
2. ETL pipeline normalizes records into a common schema, decodes function signatures, and resolves contract addresses to logical application entities (e.g., multiple Railgun contracts become one Application node).
3. Feature engineering jobs compute node-level statistics: in/out degree, balance windows, activity timezone distribution, gas fee patterns, same-amount transfer ratios.
4. Nodes and edges are upserted into Neo4j. Known labels from Etherscan/Allium/OFAC are synced to both Neo4j node properties and PostgreSQL reference tables.
5. Analysts interact via the frontend: explore subgraphs, run path queries, apply labels. Labels are stored in PostgreSQL with full audit trails.
6. Graph algorithms (PageRank, community detection, centrality) run periodically via Neo4j GDS to surface high-risk clusters and intermediary nodes.

3. Data Sources

3.1 Primary: Allium

Allium is the recommended primary data source. It provides pre-decoded, indexed blockchain data accessible via standard SQL. Key advantages for AML analysis:

- Internal transactions (traces) are pre-indexed — essential for tracing funds through contract-to-contract calls that are invisible in normal transaction tables.
- Decoded events and function calls eliminate the need for ABI resolution and input data parsing.
- Cross-chain support enables future expansion beyond Ethereum.
- Pre-tagged contract labels reduce bootstrapping effort for known entity identification.

3.2 Key Allium Tables

Table	Description	AML Relevance
ethereum.transactions	All top-level transactions	Primary transfer records, gas analysis
ethereum.traces	Internal transactions (call traces)	Critical: reveals contract-to-contract fund flows hidden from surface txs
ethereum.decoded_events	ABI-decoded event logs	Token transfers (ERC20 Transfer events), DEX swaps, protocol interactions
ethereum.contracts	Deployed contract metadata	Contract deployer identification, creation timestamps
ethereum.token_transfers	Normalized ERC20/721/1155 transfers	Simplified token flow analysis without manual event decoding

3.3 Fallback Sources

Source	Use Case	Limitations
Etherscan API	Ad-hoc lookups, ABI fetching, label verification	5 req/sec free tier, no bulk internal txs on free plan
Self-hosted Erigon	Full historical replay, zero rate limits, raw trace access	2TB+ storage, weeks to sync, manual trace decoding required

4. Database Schema

4.1 Neo4j Graph Schema

4.1.1 Node Labels (Entity Types)

All entities carry the base label `:Entity`. Secondary labels provide type-specific classification. Neo4j supports multi-labeling, so a node can be both `:Contract` and `:Mixer` simultaneously.

Label	Description	Examples
<code>:EOA</code>	Externally Owned Account (human wallet)	User wallets, bot operators
<code>:Contract</code>	Smart contract (base type)	Any deployed contract
<code>:Mixer</code>	Privacy/mixing protocol contract	Tornado Cash Router, Railgun Relay
<code>:LendingPool</code>	Lending protocol pool contract	Aave aToken pools, Compound cToken
<code>:Bridge</code>	Cross-chain bridge contract	Wormhole, Stargate, Across
<code>:DEX</code>	Decentralized exchange contract	Uniswap Router, Curve pools
<code>:CEXHotWallet</code>	Known centralized exchange wallet	Binance Hot Wallet, Coinbase
<code>:Application</code>	Logical grouping of related contracts	Railgun (all contracts as one entity)

4.1.2 Node Properties

Entity nodes carry three categories of properties: identity, computed features (materialized by ETL), and human-assigned labels.

Identity Properties

Property	Type	Description
<code>address</code>	String	Checksummed Ethereum address (primary key, unique constraint)
<code>entity_type</code>	String	<code>EOA</code> <code>contract</code> (redundant with label, useful for filtered queries)
<code>first_seen_at</code>	DateTime	Timestamp of first observed transaction
<code>last_seen_at</code>	DateTime	Timestamp of most recent transaction
<code>is_contract</code>	Boolean	Whether address has deployed bytecode
<code>deployed_by</code>	String	Creator address (if contract)

Computed Feature Properties (ETL-materialized)

Property	Type	Description
tx_count_in	Integer	Total incoming transfer count
tx_count_out	Integer	Total outgoing transfer count
unique_counterparties_in	Integer	Distinct addresses that sent funds to this entity
unique_counterparties_out	Integer	Distinct addresses this entity sent funds to
balance_current	Float	Current ETH balance
balance_avg_30d	Float	30-day rolling average balance
balance_max_30d	Float	30-day maximum balance
active_hours	List<Int>	Activity histogram by UTC hour (0-23)
median_tx_interval	Float	Median seconds between outgoing transactions
gas_price_avg	Float	Average gas price used
gas_price_stddev	Float	Gas price standard deviation
same_amount_out_count	Integer	Outgoing txs with identical value
same_amount_out_ratio	Float	Ratio of same-amount txs to total outgoing
same_type_target_count	Integer	Transfers to entities of same type
has_deployed_contracts	Boolean	Whether this EOA has deployed any contracts

Label Properties

Property	Type	Description
known_label	String	Human-readable identity (e.g., Binance Hot Wallet 1)
label_source	String	etherscan allium ofac manual
risk_label	String	clean suspicious flagged unlabeled
risk_score	Float	0.0 – 1.0, computed by model or analyst
labeled_by	String	Annotator identifier
labeled_at	DateTime	When label was applied

4.1.3 Application (Macro Node)

Application nodes group multiple contracts into a single logical entity. This supports both macro-level analysis (all money flowing into Railgun) and micro-level inspection (which specific Railgun contract received the most).

Property	Type	Description
app_id	String	Unique identifier (e.g., railgun, tornado-cash, aave-v3)
name	String	Display name
category	String	mixer lending bridge dex
chain	String	Primary chain (ethereum, polygon, etc.)
risk_tier	String	high medium low

Relationship: (:Contract)-[:BELONGS_TO]->(:Application)

4.1.4 Edge (Relationship) Types

TRANSFER — Value transfer between entities

Property	Type	Description
tx_hash	String	Transaction hash (unique identifier)
block_number	Integer	Block in which transaction was mined
timestamp	DateTime	Block timestamp
value	Float	Transfer amount in ETH (or wei as String for precision)
token_address	String	Null for native ETH, otherwise ERC20 contract address
token_symbol	String	Token ticker symbol
gas_used	Integer	Gas consumed by transaction
gas_price	Float	Gas price in Gwei
function_sig	String	4-byte selector or decoded signature
is_internal	Boolean	Whether from trace (not top-level tx)
trace_address	String	Nested call position (e.g., 0.1.0)
log_index	Integer	Event log index within transaction

CALLS — Non-value contract interaction

Property	Type	Description
tx_hash	String	Transaction hash
timestamp	DateTime	Block timestamp
function_sig	String	Called function signature

gas_used	Integer	Gas consumed
----------	---------	--------------

DEPLOYED — Contract creation

Property	Type	Description
tx_hash	String	Deployment transaction hash
timestamp	DateTime	Deployment timestamp
block_number	Integer	Deployment block

4.1.5 Indexes**Required indexes for query performance:**

```

CREATE CONSTRAINT entity_address FOR (e:Entity) REQUIRE e.address IS UNIQUE;

CREATE INDEX entity_type FOR (e:Entity) ON (e.entity_type);

CREATE INDEX entity_risk FOR (e:Entity) ON (e.risk_label);

CREATE INDEX entity_last_seen FOR (e:Entity) ON (e.last_seen_at);

CREATE INDEX transfer_timestamp FOR ()-[t:TRANSFER]-() ON (t.timestamp);

CREATE INDEX transfer_value FOR ()-[t:TRANSFER]-() ON (t.value);

CREATE INDEX transfer_token FOR ()-[t:TRANSFER]-() ON (t.token_address);

CREATE FULLTEXT INDEX entity_search FOR (e:Entity) ON EACH [e.address, e.known_label];

```

4.2 PostgreSQL Schema

PostgreSQL handles labeling workflows, user management, known entity references, and ETL job tracking. This data is relational in nature and benefits from ACID guarantees and standard SQL tooling.

4.2.1 labelers

Column	Type	Constraints	Description
id	UUID	PK, auto-generated	Unique labeler identifier
username	TEXT	UNIQUE, NOT NULL	Login username
role	TEXT	NOT NULL, CHECK IN (admin, labeler, reviewer)	Access level
created_at	TIMESTAMPTZ	DEFAULT now()	Account creation time

4.2.2 label_tasks

Column	Type	Constraints	Description
id	UUID	PK	Task identifier
address	TEXT	NOT NULL	Target entity address to label
subgraph_depth	INTEGER	DEFAULT 2	Hops to show in annotation UI
priority	INTEGER	DEFAULT 0	Task priority for queue ordering
status	TEXT	CHECK IN (pending, assigned, completed, disputed)	Workflow state
assigned_to	UUID	FK → labelers(id)	Currently assigned annotator
created_at	TIMESTAMPTZ	DEFAULT now()	Task creation time
completed_at	TIMESTAMPTZ	Nullable	Task completion time

4.2.3 annotations

Column	Type	Constraints	Description
id	UUID	PK	Annotation identifier
task_id	UUID	FK → label_tasks(id)	Parent task
labeler_id	UUID	FK → labelers(id)	Annotator
address	TEXT	NOT NULL	Annotated entity address
verdict	TEXT	CHECK IN (clean, suspicious, laundering, uncertain)	Classification decision
confidence	FLOAT	CHECK 0–1	Annotator confidence score

evidence	JSONB	Nullable	Free-form evidence (links, notes, screenshots)
tags	TEXT[]	Nullable	Pattern tags: structuring, layering, peel-chain, round-amount
created_at	TIMESTAMPTZ	DEFAULT now()	Annotation timestamp

4.2.4 known_labels

Column	Type	Constraints	Description
address	TEXT	PK	Entity address
label	TEXT	NOT NULL	Human-readable label
category	TEXT	Nullable	exchange mixer defi bridge
source	TEXT	NOT NULL	etherscan allium ofac manual
risk_tier	TEXT	DEFAULT unknown	high medium low unknown
updated_at	TIMESTAMPTZ	DEFAULT now()	Last update time

4.2.5 ingestion_runs

Column	Type	Constraints	Description
id	UUID	PK	Run identifier
source	TEXT	NOT NULL	allium erigon etherscan
block_start	BIGINT	Nullable	Start block of ingestion range
block_end	BIGINT	Nullable	End block of ingestion range
entity_count	INTEGER	Nullable	Nodes created/updated
edge_count	INTEGER	Nullable	Edges created/updated
status	TEXT	DEFAULT running	running completed failed
started_at	TIMESTAMPTZ	DEFAULT now()	Job start time
completed_at	TIMESTAMPTZ	Nullable	Job completion time
error	TEXT	Nullable	Error message if failed

4.2.6 label_consensus (View)

Materialized view for inter-annotator agreement metrics. Aggregates annotations by address to determine consensus verdict, annotator count, verdict diversity, and average confidence. Used for quality assurance and identifying disputed entities requiring reviewer escalation.

5. Graph Algorithms

Neo4j GDS (Graph Data Science) library provides the algorithmic backbone for pattern detection. These algorithms run as batch jobs on projected subgraphs, with results written back to node/relationship properties.

Algorithm	Category	AML Application
PageRank	Centrality	Identify high-influence nodes that act as major fund aggregators or distributors
Betweenness Centrality	Centrality	Detect intermediary/mixer nodes that sit on many shortest paths between flagged entities
Louvain	Community Detection	Cluster wallets into communities; suspicious clusters often share behavioral patterns
Label Propagation	Community Detection	Fast community assignment for large graphs; seed with known labels to propagate risk
Weakly Connected Components	Community Detection	Isolate disconnected subgraphs; small components may indicate layering networks
Shortest Path / All Paths	Pathfinding	Trace fund flows from flagged source to known destination (exchange deposit)
Node Similarity	Similarity	Find wallets with similar behavioral fingerprints (timing, amounts, counterparties)

5.1 Custom AML Queries

Beyond standard graph algorithms, the following Cypher patterns detect AML-specific behaviors:

- Peel chain detection: sequential transfers of decreasing amounts from a single source through intermediate wallets to a final destination.
- Structuring detection: multiple transfers of near-identical amounts within a time window from the same source.
- Round-trip detection: funds that return to the same entity after passing through intermediaries (layering).
- Fan-out / fan-in patterns: single entity distributing to many wallets that later consolidate into one destination.
- Timing correlation: entities that transact within narrow time windows of each other, suggesting coordination.

6. Technology Stack

Component	Technology	Rationale
Graph Database	Neo4j 5.x + GDS Plugin	Native graph storage, Cypher query language, built-in graph algorithms
Relational Database	PostgreSQL 16	ACID for labeling workflows, JSONB for flexible evidence storage
Message Queue	Kafka or Redis Streams	Decouple ingestion rate from graph write rate
ETL Orchestration	Dagster (Python)	Asset-based orchestration, built-in observability, retry handling
Backend API	FastAPI (Python)	Async support, auto-generated OpenAPI docs, Pydantic validation
Frontend Framework	React 18 + TypeScript	Component ecosystem, strong typing, wide adoption
Graph Visualization	Cytoscape.js	Best balance of features, performance, and graph-specific layouts
UI Components	shadcn/ui + Tailwind CSS	Accessible components, rapid prototyping
Charts	Recharts	React-native charting for dashboards and analytics panels
Auth	JWT (simple) or Supabase Auth	Lightweight for internal tools; Supabase if multi-team
Containerization	Docker Compose (dev), K8s (prod)	Reproducible environments, scaling for Neo4j GDS workloads
Data Source	Allium (primary)	Pre-decoded traces, SQL interface, cross-chain ready

7. Development Guide

7.1 Project Structure

```

chain-analysis/
├── docker-compose.yml          # Neo4j, Postgres, Kafka, API, Frontend
├── .env.example                # Environment variables template
├── backend/
│   ├── pyproject.toml          # Python deps (poetry/uv)
│   ├── src/
│   │   ├── api/                # FastAPI routes
│   │   │   ├── routes/          # Endpoint modules
│   │   │   ├── models/          # Pydantic request/response models
│   │   │   └── deps.py          # Dependency injection (DB sessions)
│   │   ├── etl/                # Dagster pipeline definitions
│   │   │   ├── assets/          # Dagster assets (ingest, transform, load)
│   │   │   ├── resources/       # DB connections, API clients
│   │   │   └── jobs.py          # Scheduled job definitions
│   │   ├── graph/              # Neo4j query builders + GDS wrappers
│   │   ├── db/                 # SQLAlchemy models + migrations (Alembic)
│   │   └── services/            # Business logic layer
│   └── tests/
├── frontend/
│   ├── package.json
│   ├── src/
│   │   ├── components/         # React components
│   │   │   ├── graph/          # Cytoscape wrapper, node/edge renderers
│   │   │   ├── labeling/       # Annotation UI, verdict forms
│   │   │   └── dashboard/      # Analytics charts, summary views
│   │   ├── hooks/              # Custom React hooks (useGraph, useLabel)
│   │   └── api/                # API client (fetch wrappers, types)

```

```
| | └─ types/                # TypeScript interfaces
| └─ public/
└─ scripts/                  # Utility scripts (seed data, migrations)
```

7.2 Local Development Setup

Prerequisites

- Docker Desktop with at least 8GB RAM allocated (Neo4j GDS is memory-hungry)
- Node.js 20+ and pnpm
- Python 3.12+ with uv or poetry
- Allium account (free tier sufficient for development)

Step-by-Step Setup

1. Clone the repository and copy `.env.example` to `.env`. Fill in Allium API key and Neo4j/Postgres credentials.
2. Run `docker compose up -d` to start Neo4j (with GDS plugin), PostgreSQL, and Kafka.
3. Run `alembic upgrade head` from `backend/` to initialize PostgreSQL schema.
4. Run the seed script (`python scripts/seed_known_labels.py`) to populate `known_labels` from Etherscan/Allium tag exports.
5. Start the Dagster UI with `dagster dev` from `backend/` to monitor and trigger ETL pipelines.
6. Run the initial ingestion job targeting a small block range (e.g., 1000 blocks) to verify the pipeline.
7. Start the FastAPI server: `uvicorn src.api.main:app --reload` from `backend/`.
8. Start the frontend: `pnpm dev` from `frontend/`. The app will connect to the API at `localhost:8000`.

7.3 Docker Compose Configuration

```
services:
```

```
  neo4j:
```

```
    image: neo4j:5.x-enterprise
```

```
    environment:
```

```
      - NEO4J_ACCEPT_LICENSE_AGREEMENT=yes
      - NEO4J_PLUGINS=["graph-data-science"]
      - NEO4J_AUTH=neo4j/${NEO4J_PASSWORD}
      - NEO4J_server_memory_heap_max_size=4G
      - NEO4J_server_memory_pagecache_size=2G
```

```
    ports: ["7474:7474", "7687:7687"]
```

```
volumes: ["neo4j_data:/data"]
```

```
postgres:
```

```
image: postgres:16-alpine
```

```
environment:
```

- POSTGRES_DB=chain-analysis
- POSTGRES_USER=\${PG_USER}
- POSTGRES_PASSWORD=\${PG_PASSWORD}

```
ports: ["5432:5432"]
```

```
volumes: ["pg_data:/var/lib/postgresql/data"]
```

```
kafka:
```

```
image: confluentinc/cp-kafka:7.x
```

```
# ... standard Kafka config
```

7.4 ETL Pipeline Development

Dagster Asset Structure

The ETL pipeline is organized as Dagster assets with clear dependencies:

Asset	Upstream	Description
raw_transactions	Allium API	Fetch transactions for block range, write to staging
raw_traces	Allium API	Fetch internal transactions (traces)
raw_events	Allium API	Fetch decoded events
resolved_entities	raw_transactions, raw_traces	Merge addresses, identify contracts vs EOAs, group into applications
computed_features	resolved_entities, raw_transactions	Calculate node-level stats (degree, balance, timing, patterns)
graph_nodes	resolved_entities, computed_features	Upsert Entity nodes into Neo4j via MERGE
graph_edges	raw_transactions, raw_traces, graph_nodes	Upsert TRANSFER/CALLS/DEPLOYED relationships into Neo4j
sync_labels	known_labels (Postgres)	Sync known labels from Postgres to Neo4j node properties
gds_algorithms	graph_nodes, graph_edges	Run PageRank, Louvain, centrality on projected subgraph

Key Implementation Notes

- Use Neo4j MERGE (not CREATE) for idempotent upserts. This allows re-running ingestion for the same block range without duplicates.
- Batch Neo4j writes using UNWIND for performance. Individual CREATE/MERGE statements are 10-100x slower than batched UNWIND operations.
- Use Allium's block_number range filters to enable incremental ingestion. Track the last processed block in ingestion_runs.
- Feature computation should be a separate Dagster asset that runs after node/edge creation, not inline during ingestion.
- For token amounts, store raw wei values as strings to avoid floating-point precision loss. Convert to human-readable amounts in the API/frontend layer.

7.5 API Development

Core Endpoints

Method	Path	Description
GET	/api/entities/{address}	Get entity node with all properties

GET	/api/entities/{address}/neighbors	Get immediate neighbors (1-hop subgraph)
GET	/api/entities/{address}/paths/{target}	Find all paths between two addresses
POST	/api/graph/query	Execute custom Cypher query (admin only)
GET	/api/graph/communities	List detected communities with risk scores
POST	/api/labels/tasks	Create labeling task for an address
GET	/api/labels/tasks?status=pending	List pending labeling tasks
POST	/api/labels/annotations	Submit annotation for a task
GET	/api/labels/consensus/{address}	Get consensus label for address
GET	/api/analytics/risk-distribution	Aggregate risk score distribution
GET	/api/analytics/ingestion-status	Latest ETL run status

Neo4j Driver Pattern

Use the async Neo4j Python driver with session-per-request pattern:

```
from neo4j import AsyncGraphDatabase
```

```
driver = AsyncGraphDatabase.driver(NEO4J_URI, auth=(user, password))
```

```
async def get_neighbors(address: str, depth: int = 1):
    query = """
    MATCH (e:Entity {address: $address})-[t:TRANSFER*1..{depth}]- (n:Entity)
    RETURN e, t, n LIMIT 200
    """
    async with driver.session() as session:
        result = await session.run(query, address=address, depth=depth)
        return [record.data() async for record in result]
```

7.6 Frontend Development

Graph Visualization with Cytoscape.js

The graph component is the core of the analyst experience. Key implementation decisions:

- Lazy loading: fetch only the immediate neighborhood of a clicked node (1-2 hops). Never load the entire graph at once.
- Layout: use fcose (force-directed with compound node support) as the default layout. Allow users to switch to hierarchical (dagre) for flow analysis.
- Styling: color-code nodes by entity_type and risk_label. Use edge thickness to represent transfer value (log scale).
- Interaction: click node to expand neighbors, right-click for context menu (label, flag, find paths to), hover for tooltip with key properties.
- Performance: for subgraphs exceeding 5,000 nodes, switch to WebGL renderer (Sigma.js) or implement server-side pagination.

Frontend Component Hierarchy

Component	Responsibility
<App>	Router, auth context, global layout
<GraphExplorer>	Main investigation view: graph canvas + control panel
<GraphCanvas>	Cytoscape.js wrapper, handles layout/styling/events
<NodePanel>	Side panel showing selected node details + label form
<PathFinder>	Input two addresses, visualize all paths between them
<FilterBar>	Filter by entity type, risk level, time range, amount range
<LabelQueue>	Task list for labeling assignments
<AnnotationForm>	Verdict, confidence, evidence, tags input
<Dashboard>	Summary analytics: risk distribution, ingestion status, community overview

8. Development Phases

Phase 1: Foundation (Weeks 1–3)

- Docker Compose setup: Neo4j (with GDS), PostgreSQL, base API container.
- PostgreSQL schema: run Alembic migrations for labelers, label_tasks, annotations, known_labels, ingestion_runs.
- Neo4j schema: create constraints, indexes, and fulltext search index.
- Basic ETL: Allium integration for a single block range → Neo4j nodes + edges. Manual trigger only.
- Seed known_labels with Etherscan tag export and OFAC sanctioned addresses.
- API scaffolding: entity lookup, neighbor expansion, basic CRUD for labels.

Phase 2: Core Analysis (Weeks 4–6)

- Feature engineering pipeline: compute all node-level stats (degree, balance windows, timing, gas patterns).
- GDS integration: run PageRank, Louvain, betweenness centrality as scheduled Dagster jobs.
- Path query endpoints: shortest path, all paths, with depth and time-range constraints.
- Custom AML Cypher queries: peel chain detection, structuring, fan-out/fan-in patterns.
- Incremental ingestion: track last processed block, run ETL on schedule (e.g., every 100 blocks).

Phase 3: Frontend & Labeling (Weeks 7–10)

- Graph explorer: Cytoscape.js canvas with lazy neighbor loading, node styling by type/risk.
- Node detail panel: display all properties, transaction history, linked application.
- Labeling workflow: task queue, annotation form (verdict, confidence, evidence, tags), consensus view.
- Filter and search: fulltext search by address/label, filter by risk, entity type, time range.
- Path visualization: highlight paths between two addresses with transaction details on edges.
- Dashboard: risk distribution charts, ingestion pipeline health, community summaries.

Phase 4: Hardening & Scale (Weeks 11+)

1. Performance: Neo4j query optimization (profile + explain), index tuning, batch write optimization.
2. Data retention: archive old edges to PostgreSQL/DuckDB, keep active analysis window in Neo4j (e.g., 90 days).
3. Multi-chain: extend Allium queries to Polygon, Arbitrum, BSC. Add chain property to all nodes/edges.
4. Model integration: train GNN or classifier on labeled dataset, write risk_score back to nodes.
5. Export: generate investigation reports (PDF/DOCX) from labeled subgraphs for compliance teams.

9. Operational Considerations

9.1 Neo4j Memory Planning

Neo4j GDS projects the graph into memory for algorithm execution. A rough estimate for Ethereum mainnet:

Scale	Nodes	Edges	GDS Memory Required
Dev / POC	~100K	~500K	2–4 GB
Single protocol analysis	~1M	~10M	8–16 GB
Broad Ethereum coverage	~10M	~100M	64–128 GB

Recommendation: start with a focused subgraph (e.g., all addresses within 3 hops of known mixer contracts) rather than ingesting all of Ethereum. Expand scope as the labeling dataset grows and analytical needs become clearer.

9.2 Data Freshness vs. Cost

Allium free tier has query limits. For development, batch ingestion every few hours is sufficient. For production analysis of active laundering cases, consider upgrading to Allium's paid tier or supplementing with a self-hosted Erigon node for real-time trace access.

9.3 Security

- Neo4j and PostgreSQL should not be exposed to the public internet. Use Docker internal networking + API gateway.
- The `/api/graph/query` endpoint (raw Cypher execution) must be restricted to admin roles only.
- Labeling data may contain sensitive intelligence. Implement row-level security in PostgreSQL if multi-team access is required.
- Allium API keys should be stored in environment variables or a secrets manager, never committed to source control.

10. Hybrid Rust/Python Architecture

At 5TB+ of transaction data, a pure Python stack becomes a bottleneck in the ETL and feature computation layers. However, rewriting the entire backend in Rust yields diminishing returns for I/O-bound components like the API server and labeling CRUD. The recommended approach is a hybrid architecture that applies Rust where CPU and memory performance are critical, and retains Python where developer productivity and ecosystem access matter more.

10.1 Where Rust Wins

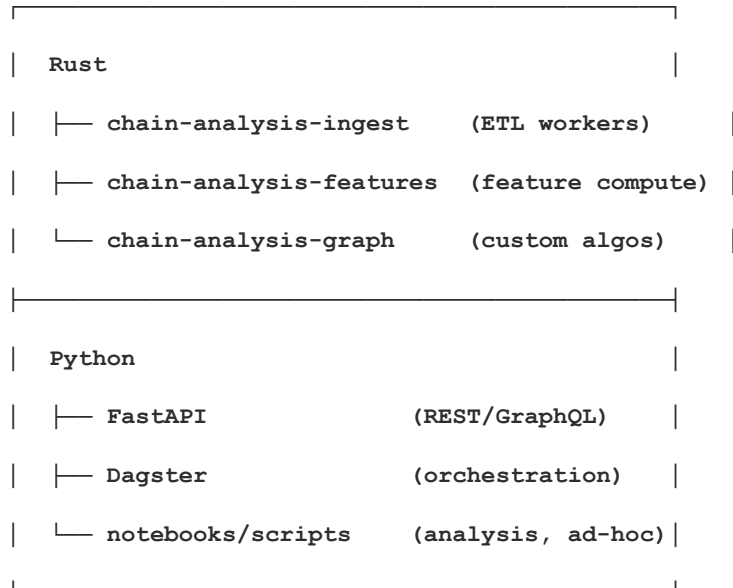
Component	Why Rust	Expected Improvement
ETL Ingestion Workers	Decoding billions of transactions, ABI resolution, entity merging. Python GIL and memory overhead are the primary bottleneck.	10–50x throughput, 5–10x less memory
Feature Computation	Computing node-level stats (degree, balance windows, timing patterns) across the full dataset. Numerically intensive with large intermediate data.	10–30x faster batch processing
Custom Graph Algorithms	If Neo4j GDS cannot fit the full graph in memory, out-of-core traversals with petgraph or custom CSR representation. Memory-mapped graph processing that Python cannot do.	Enables analysis at scales impossible in Python
Stream Processing	Real-time ingestion from Ethereum node (consuming blocks as produced). Backpressure handling and sustained throughput for Kafka consumers.	Stable throughput under load, lower tail latency

10.2 Where Python Stays

Component	Why Python	Rationale
API Server (FastAPI)	I/O bound, waiting on DB queries	Async Neo4j driver handles concurrency. Axum would be marginally faster but the bottleneck is database round-trips, not the web framework.
ETL Orchestration (Dagster)	Scheduling, retries, dependency graph	Dagster's Python SDK is mature. It triggers Rust workers as external processes or Kafka consumers.
Labeling System CRUD	Pure Postgres reads/writes	Language choice is irrelevant for simple CRUD operations.
Ad-hoc Analysis & Notebooks	Exploratory Cypher queries, data inspection	Python iteration speed and Jupyter ecosystem are unmatched for analysis.

10.3 Inter-Process Communication

Rust workers communicate with the Python orchestration layer via message queues. Dagster triggers Rust binaries as external processes or publishes tasks to Kafka/SQS topics that Rust consumers pick up. Results flow back through the same queue or are written directly to Neo4j/PostgreSQL.



11. Cloud Deployment (AWS)

For production deployment, managed cloud services replace self-hosted Docker containers to reduce operational overhead, improve availability, and enable elastic scaling. The following maps each self-managed component to its AWS equivalent.

11.1 Service Mapping

Component	Self-Managed	AWS Service	Notes
Relational DB	PostgreSQL	RDS PostgreSQL or Aurora PostgreSQL	Aurora for auto-scaling reads. RDS is cheaper for labeling workloads.
Graph DB	Neo4j (Docker)	Neo4j Aura (AWS Marketplace)	Full Cypher + GDS support. See Neptune comparison below.
Message Queue	Kafka	MSK (Managed Kafka) or SQS	SQS is simpler and cheaper if strict ordering is not required.
ETL Orchestration	Dagster (self-hosted)	Dagster on ECS or Step Functions + Lambda	Dagster on ECS preferred — better for data pipelines than Step Functions.
Rust ETL Workers	Docker containers	ECS Fargate	Fargate for long-running ingestion. Lambda if batches complete in <15 min.
API Server	FastAPI (Docker)	ECS Fargate or App Runner	App Runner is simpler (auto-scaling, no infra config). Fargate for VPC control.
Frontend	React dev server	S3 + CloudFront	Static hosting, globally cached CDN. Cheapest option.
Cold Storage	DuckDB / Parquet files	S3 + Athena	Athena = serverless SQL over Parquet on S3. Perfect for 5TB cold tier.
Auth	JWT / Supabase	Cognito	Sufficient for internal tools. Annoying to customize but functional.
Secrets	.env files	Secrets Manager or SSM Parameter Store	SSM is free. Secrets Manager adds rotation features.
Monitoring	Dagster UI + logs	CloudWatch + X-Ray	X-Ray for distributed tracing across ECS services.

11.2 Neptune vs. Neo4j Aura

The graph database choice is the most consequential decision for the cloud deployment. Both options are viable, but they differ significantly in algorithm support.

Amazon Neptune

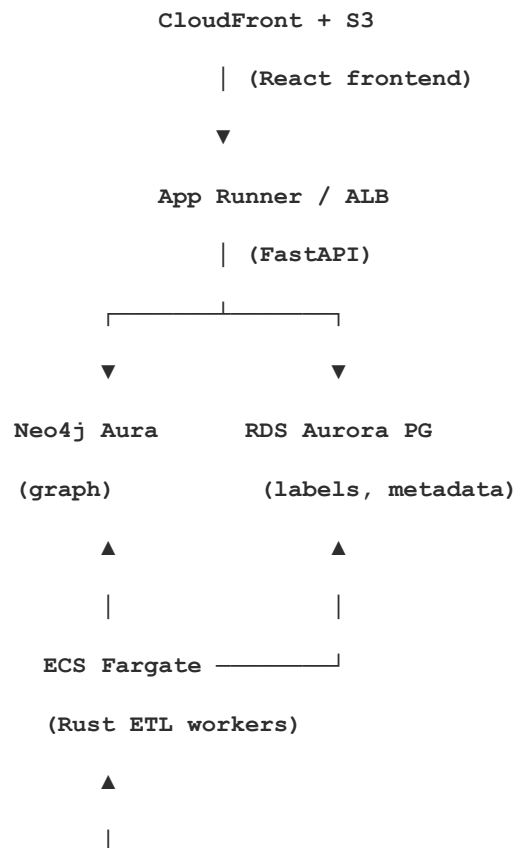
- Fully managed, no licensing overhead. Auto-scaling storage.
- Supports openCypher (subset of Neo4j Cypher) and Gremlin query languages.
- No equivalent to Neo4j GDS — no built-in PageRank, Louvain, community detection, or centrality algorithms.
- Neptune Analytics is a newer offering with limited graph algorithm support, but not yet mature.
- Choosing Neptune means implementing graph algorithms in Rust (petgraph) or Python (NetworkX/igraph) outside the database.

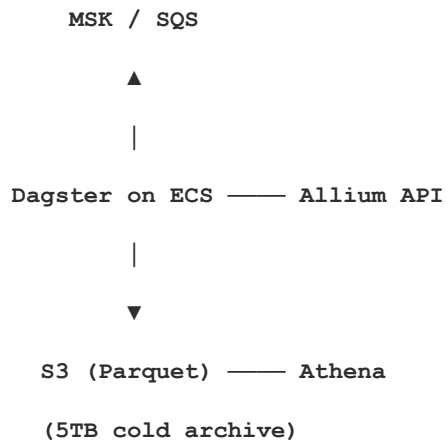
Neo4j Aura (on AWS Marketplace)

- Full Cypher query language with complete GDS (Graph Data Science) library.
- All graph algorithms (PageRank, Louvain, betweenness centrality, node similarity) work out of the box.
- More expensive than Neptune at equivalent scale (Neo4j licensing + AWS infrastructure).
- Professional tier required for GDS — free tier does not include graph algorithms.

Recommendation: Neo4j Aura Professional. The GDS algorithms are core to the AML detection pipeline. Reimplementing PageRank and Louvain externally defeats the purpose of using a graph database and adds significant development and maintenance burden.

11.3 AWS Architecture Diagram





11.4 Cost Estimate (Monthly)

Approximate monthly costs for a production-grade deployment handling the full 5TB dataset with active investigation workloads:

Service	Configuration	Estimated Cost
Neo4j Aura Professional	16 GB RAM, 64 GB storage	\$700 – \$1,200
RDS Aurora PostgreSQL	db.r6g.large (2 vCPU, 16 GB)	\$200 – \$300
ECS Fargate (ETL Workers)	2 vCPU, 8 GB, burst usage	\$100 – \$200
ECS Fargate / App Runner (API)	1 vCPU, 2 GB, always-on	\$50 – \$100
S3 (5 TB Standard)	Standard tier storage	~\$115
Athena	Per-query pricing, ~200 queries/month	\$50 – \$100
MSK (if used)	kafka.m5.large × 2 brokers	~\$300
Total (with MSK)		\$1,500 – \$2,400
Total (with SQS instead)		\$1,200 – \$2,100

SQS instead of MSK saves approximately \$300/month. Recommended unless strict message ordering is required for the ETL pipeline (it typically is not for batch ingestion).

11.5 GCP Alternative

If deploying on Google Cloud Platform instead of AWS, the service mapping shifts as follows:

AWS Service	GCP Equivalent	Notes
RDS / Aurora	Cloud SQL for PostgreSQL	Similar managed Postgres offering
S3	Google Cloud Storage (GCS)	Allium can export directly to GCS
Athena	BigQuery	Better than Athena for cold-tier analytics. Columnar, serverless.
ECS Fargate	Cloud Run	Simpler container deployment, auto-scaling to zero
MSK / SQS	Pub/Sub	Fully managed, no broker provisioning
CloudFront + S3	Cloud CDN + GCS	Static frontend hosting
Cognito	Firebase Auth or Identity Platform	More flexible than Cognito
CloudWatch	Cloud Monitoring + Cloud Trace	Equivalent observability stack

Neo4j Aura is available on GCP Marketplace with the same GDS capabilities. The graph database choice remains the same regardless of cloud provider.

12. Tiered Storage Strategy

At 5TB of raw transaction data, loading everything into Neo4j is neither feasible nor cost-effective. The recommended approach is a three-tier storage architecture that keeps Neo4j lean for interactive analysis while the full dataset lives in cheaper, purpose-appropriate storage.

Tier	Storage	Data	Purpose	Retention
Hot	Neo4j Aura	Active investigation subgraphs: flagged entities + N-hop neighborhoods	Interactive graph queries, GDS algorithm execution, real-time exploration	Dynamic (promoted/demoted on demand)
Warm	RDS Aurora PG (partitioned)	All transactions, partitioned by block range. Complete node feature history.	Feature computation, historical lookups, labeling workflow data	Full history
Cold	S3 Parquet + Athena	Full 5TB raw archive: transactions, traces, events	Batch analytics, ML training data, compliance audit trail	Full history, lifecycle to Glacier after 1 year

12.1 Promotion / Demotion Pipeline

A Rust-based promotion worker handles moving data between tiers:

- Promotion (Cold/Warm → Hot): when an analyst flags an address or a risk model identifies a suspicious entity, the worker pulls the N-hop subgraph from PostgreSQL or Parquet and materializes it into Neo4j as nodes + edges.
- Demotion (Hot → Warm): subgraphs that have been resolved (labeled clean or case closed) are archived out of Neo4j to reduce memory pressure. Node properties and labels are preserved in PostgreSQL.
- This keeps Neo4j at a manageable size (50–100 GB) while the full 5TB dataset remains accessible for batch queries via Athena or PostgreSQL.

12.2 Query Routing

The API layer routes queries to the appropriate tier based on the request type:

- Graph traversal, neighbor expansion, path queries, GDS algorithms → Neo4j (hot tier).
- Entity lookup by address, feature history, label CRUD → PostgreSQL (warm tier).
- Full-dataset analytics, ML feature extraction, compliance exports → Athena over S3 (cold tier).
- If a graph query references an entity not yet in Neo4j, the API triggers an async promotion job and returns a partial result with a status indicator.